AccessIndia AI

STEP-BY-STEP BUILD GUIDE

With Code, Tests & Prompts

PRD v2.0 | 26 July 2026

Team Coin Hustlers

Tanya | Moin | Kush | Areeb | Smarpit

HackAgentAIx 2026 -- Track 2: Multi-Agent Collaboration

HOW TO USE THIS GUIDE

This is your single source of truth for the 48-hour build. Follow steps in order. Each step includes: what to build, exact code, how to test it, and what the checkpoint looks like.

WORKFLOW

1. Read the step. 2. Copy the code. 3. Run the test. 4. Only when test passes, move to next step. Do NOT skip tests. A green test is your proof the step works.

Tech Stack (Locked)

- Frontend: React 18 + Vite + Tailwind CSS + Zustand + Axios
- Backend: FastAPI + Uvicorn + Google Generative AI (Gemini 1.5 Flash)
- Browser APIs: Web Speech API, MediaPipe Hands, Geolocation, Google Maps JS
- NO EasyOCR. NO YOLO. NO ElevenLabs. Gemini handles all image + text AI.

File Structure

```
accessindia/
  backend/
    app/
      __init__.py
      main.py
      config.py
      models.py
      agents/
        __init__.py
        orchestrator.py
        vision_agent.py
        navigation_agent.py
        audit_agent.py
      routers/
        __init__.py
        chat.py
        vision.py
        navigation.py
        audit.py
      utils/
        __init__.py
        prompts.py
    tests/
      test_chat.py
      test_vision.py
      test_navigation.py
      test_audit.py
      conftest.py
    requirements.txt
    .env
    pytest.ini
  frontend/
    src/
      main.jsx
      App.jsx
      store/
        useAppStore.js
      components/
        Layout/
          Sidebar.jsx
          Header.jsx
          ChatBubble.jsx
        Agents/
          OrchestratorChat.jsx
          VisionAgent.jsx
          CommunicationAgent.jsx
```

```
      NavigationAgent.jsx
      AuditAgent.jsx
    Shared/
      CameraFeed.jsx
      FileDrop.jsx
      TTSButton.jsx
      LoadingAgent.jsx
  hooks/
    useSpeechToText.js
    useTextToSpeech.js
    useMediaPipe.js
    useGeolocation.js
  services/
    api.js
  utils/
    gestureClassifier.js
tests/
  App.test.jsx
  useSpeechToText.test.js
  gestureClassifier.test.js
index.html
vite.config.js
tailwind.config.js
postcss.config.js
package.json
```

0 PRE-HACKATHON SETUP [Before July 30]

0.1 Get API Keys

- Gemini API Key: https://aistudio.google.com/app/apikey
- Google Maps API Key: https://console.cloud.google.com/
- Enable APIs: Maps JavaScript API, Directions API, Places API

0.2 Create Project Directories

```
mkdir accessindia && cd accessindia
mkdir backend frontend
cd backend
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
```

0.3 Backend Dependencies

```
pip install fastapi uvicorn python-multipart google-generativeai
pip install pydantic-settings python-dotenv sqlalchemy pytest httpx
pip install pytest-asyncio

# Save to requirements.txt
pip freeze > requirements.txt
```

0.4 Frontend Dependencies

```
cd ../frontend
npm create vite@latest . -- --template react
npm install
npm install tailwindcss postcss autoprefixer
npx tailwindcss init -p
npm install zustand axios framer-motion lucide-react react-router-dom
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

0.5 Environment File

```
# backend/.env
GEMINI_API_KEY=your_gemini_key_here
GOOGLE_MAPS_API_KEY=your_maps_key_here
CORS_ORIGINS=http://localhost:5173,https://your-vercel-app.vercel.app
BACKEND_PORT=8000

# frontend/.env
VITE_API_URL=http://localhost:8000
VITE_GOOGLE_MAPS_KEY=your_maps_key_here
```

CHECKPOINT

Run: python -c "import google.generativeai; print(OK)" and npm run dev. Both should start without errors.

1 BACKEND SCAFFOLD [30 min]

1.1 config.py

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    GEMINI_API_KEY: str
    GOOGLE_MAPS_API_KEY: str
    CORS_ORIGINS: str = "http://localhost:5173"
    BACKEND_PORT: int = 8000
    class Config:
        env_file = ".env"

@lru_cache()
def get_settings():
    return Settings()

settings = get_settings()
```

1.2 models.py

```
from pydantic import BaseModel, Field
from typing import List, Optional, Literal

class ChatRequest(BaseModel):
    message: str = Field(..., max_length=1000)
    session_id: Optional[str] = None

class ChatResponse(BaseModel):
    intent: Literal["VISION","COMMUNICATION","NAVIGATION","AUDIT","GENERAL"]
    agent: str
    confidence: float = Field(..., ge=0, le=1)
    message: str
    requires_upload: bool = False

class VisionResponse(BaseModel):
    ocr_text: str
    description: str
    detected_items: List[str]
    confidence: float

class NavRouteRequest(BaseModel):
    origin_lat: float = Field(..., ge=-90, le=90)
    origin_lng: float = Field(..., ge=-180, le=180)
    destination: str
    mode: str = "walking"

class NavRouteResponse(BaseModel):
    distance: str
    duration: str
    steps: List[dict]
    polyline: str

class AuditResponse(BaseModel):
    score: int = Field(..., ge=0, le=100)
    issues: List[dict]
    fixes: List[dict]
```

1.3 main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import settings
from app.routers import chat, vision, navigation, audit

app = FastAPI(
```

```
    title="AccessIndia AI",
    description="Multi-agent accessibility platform",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(chat.router, prefix="/api/chat", tags=["chat"])
app.include_router(vision.router, prefix="/api/vision", tags=["vision"])
app.include_router(navigation.router, prefix="/api/nav", tags=["navigation"])
app.include_router(audit.router, prefix="/api/audit", tags=["audit"])

@app.get("/health")
def health_check():
    return {"status": "ok", "service": "accessindia-ai"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="0.0.0.0",
                port=settings.BACKEND_PORT, reload=True)
```

1.4 Create empty __init__.py files in:

```
backend/app/__init__.py
backend/app/agents/__init__.py
backend/app/routers/__init__.py
backend/app/utils/__init__.py
```

1.5 Test: Backend Scaffold

```
# backend/tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture
def client():
    return TestClient(app)

# backend/tests/test_health.py
def test_health_check(client):
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

# Run: pytest backend/tests/test_health.py -v
```

CHECKPOINT

Run: pytest backend/tests/test_health.py -v. Should show 1 passed. Backend server starts with: python -m app.main

2 GEMINI CLIENT + ORCHESTRATOR [30 min]

2.1 utils/prompts.py

```
ORCHESTRATOR_PROMPT = '''You are the AccessIndia AI Orchestrator.
Classify user intent into exactly one of: VISION, COMMUNICATION, NAVIGATION, AUDIT, GENERAL.

Rules:
- If user mentions "read", "see", "image", "photo", "document", "label", "text" -> VISION
- If user mentions "speak", "talk", "deaf", "sign", "gesture", "hear" -> COMMUNICATION
- If user mentions "go to", "find", "nearby", "hospital", "route", "navigate", "where" -> NAVIGATION
- If user mentions "accessible", "audit", "score", "building", "wheelchair", "ramp" -> AUDIT
- If greeting ("hi", "hello") or unclear -> GENERAL

Respond ONLY in this JSON format (no markdown, no code blocks):
{"intent": "VISION", "agent": "vision", "confidence": 0.95, "message": "friendly response", "requires_upload": true}'''

VISION_PROMPT = '''Analyze this image for a visually impaired user.
1. Extract ALL readable text exactly as it appears.
2. Describe the scene in 2 natural sentences.
3. List main objects visible (max 10).

Respond in JSON:
{"ocr_text": "string", "description": "string", "detected_items": ["string"]}'''

AUDIT_PROMPT = '''Analyze this image of a public place for wheelchair accessibility.
Score 0-100 based on: Ramps (20), Doorways (20), Elevators (20), Pathways (20), Signage (20).
For each issue: category, severity (high/medium/low), description, fix suggestion, estimated cost (Low/Medium/High).

Respond in JSON:
{"score": 45, "issues": [{"category": "...", "severity": "...", "description": "..."}], "fixes": [{"priority": 1, "action": "...", "
```

2.2 agents/orchestrator.py

```
import json
import google.generativeai as genai
from app.config import settings
from app.utils.prompts import ORCHESTRATOR_PROMPT

genai.configure(api_key=settings.GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')

class OrchestratorAgent:
    def classify_intent(self, message: str, has_image: bool = False) -> dict:
        context = "User uploaded an image." if has_image else ""
        full_prompt = f"{ORCHESTRATOR_PROMPT}\n\n{context}\nUser: {message}"
        response = model.generate_content(full_prompt)
        text = response.text.strip()
        # Clean markdown code blocks if present
        for prefix in ["```json", "```"]:
            if text.startswith(prefix):
                text = text[len(prefix):]
        if text.endswith("```"):
            text = text[:-3]
        text = text.strip()
        result = json.loads(text)
        # Default to VISION if image uploaded and intent unclear
        if has_image and result["intent"] == "GENERAL":
            result["intent"] = "VISION"
            result["agent"] = "vision"
            result["message"] = "I'll analyze that image for you."
            result["requires_upload"] = True
        return result

orchestrator = OrchestratorAgent()
```

2.3 Test: Orchestrator

```
# backend/tests/test_chat.py
import pytest
from app.agents.orchestrator import orchestrator

def test_classify_vision_intent():
    result = orchestrator.classify_intent("I can't read this medicine label")
    assert result["intent"] == "VISION"
    assert result["agent"] == "vision"
    assert result["requires_upload"] == True
    assert 0 <= result["confidence"] <= 1

def test_classify_nav_intent():
    result = orchestrator.classify_intent("Find a hospital near me")
    assert result["intent"] == "NAVIGATION"
    assert result["agent"] == "navigation"

def test_classify_audit_intent():
    result = orchestrator.classify_intent("Is this building wheelchair accessible?")
    assert result["intent"] == "AUDIT"
    assert result["agent"] == "audit"

def test_classify_general_intent():
    result = orchestrator.classify_intent("Hello, what can you do?")
    assert result["intent"] == "GENERAL"
    assert result["agent"] == "general"

# Run: pytest backend/tests/test_chat.py -v
```

CHECKPOINT

Run pytest. All 4 tests should pass. If Gemini API fails, check your key in .env.

3 VISION AGENT (BACKEND) [45 min]

3.1 agents/vision_agent.py

```
import json
import google.generativeai as genai
from app.config import settings
from app.utils.prompts import VISION_PROMPT
from app.models import VisionResponse

genai.configure(api_key=settings.GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')

class VisionAgent:
    def analyze_image(self, image_bytes: bytes) -> VisionResponse:
        image_part = {"mime_type": "image/jpeg", "data": image_bytes}
        response = model.generate_content([VISION_PROMPT, image_part])
        text = response.text.strip()
        for prefix in ["```json", "```"]:
            if text.startswith(prefix):
                text = text[len(prefix):]
        if text.endswith("```"):
            text = text[:-3]
        text = text.strip()
        data = json.loads(text)
        return VisionResponse(
            ocr_text=data.get("ocr_text", ""),
            description=data.get("description", ""),
            detected_items=data.get("detected_items", []),
            confidence=0.90
        )

vision_agent = VisionAgent()
```

3.2 routers/vision.py

```
from fastapi import APIRouter, UploadFile, File, HTTPException
from app.agents.vision_agent import vision_agent
from app.models import VisionResponse

router = APIRouter()

@router.post("/analyze", response_model=VisionResponse)
async def analyze_image(file: UploadFile = File(...)):
    if file.content_type not in ["image/jpeg", "image/png", "image/jpg"]:
        raise HTTPException(status_code=400, detail="Only JPEG/PNG images allowed")
    contents = await file.read()
    if len(contents) > 4 * 1024 * 1024:
        raise HTTPException(status_code=400, detail="Image must be under 4MB")
    try:
        result = vision_agent.analyze_image(contents)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Analysis failed: {str(e)}")
```

3.3 Test: Vision Agent

```
# backend/tests/test_vision.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_vision_invalid_format():
    response = client.post("/api/vision/analyze",
        files={"file": ("test.txt", b"not an image", "text/plain")})
    assert response.status_code == 400
```

```
def test_vision_no_file():
    response = client.post("/api/vision/analyze")
    assert response.status_code == 422

# Integration test with real image:
# from PIL import Image
# img = Image.new('RGB', (100,100), color='red')
# img.save('test.jpg')
# response = client.post("/api/vision/analyze",
#     files={"file": ("test.jpg", open("test.jpg","rb"), "image/jpeg")})
# assert response.status_code == 200
# assert "ocr_text" in response.json()
```

CHECKPOINT

Upload a real image via curl or Postman to /api/vision/analyze. Should return JSON with ocr_text, description, detected_items.

4 NAVIGATION AGENT (BACKEND) [30 min]

4.1 agents/navigation_agent.py

```
import requests
from app.config import settings
from app.models import NavRouteRequest, NavRouteResponse

class NavigationAgent:
    def __init__(self):
        self.api_key = settings.GOOGLE_MAPS_API_KEY
        self.base_url = "https://maps.googleapis.com/maps/api"

    def get_route(self, req: NavRouteRequest) -> NavRouteResponse:
        url = f"{self.base_url}/directions/json"
        params = {
            "origin": f"{req.origin_lat},{req.origin_lng}",
            "destination": req.destination,
            "mode": req.mode,
            "key": self.api_key
        }
        resp = requests.get(url, params=params, timeout=10)
        data = resp.json()
        if data["status"] != "OK":
            # Fallback mock data for demo
            return NavRouteResponse(
                distance="2.3 km",
                duration="28 min",
                steps=[
                    {"instruction": "Head north toward Main Road", "distance": "100m"},
                    {"instruction": "Turn right at the traffic light", "distance": "500m"},
                    {"instruction": "Destination will be on your left", "distance": "1.7km"}
                ],
                polyline="mock_polyline_for_demo"
            )
        route = data["routes"][0]["legs"][0]
        steps = [{"instruction": s["html_instructions"],
                  "distance": s["distance"]["text"]}
                 for s in route["steps"]]
        return NavRouteResponse(
            distance=route["distance"]["text"],
            duration=route["duration"]["text"],
            steps=steps,
            polyline=data["routes"][0]["overview_polyline"]["points"]
        )

    def get_nearby(self, lat: float, lng: float, radius: int = 2000,
                   type_: str = "hospital") -> list:
        url = f"{self.base_url}/place/nearbysearch/json"
        params = {
            "location": f"{lat},{lng}",
            "radius": radius,
            "type": type_,
            "key": self.api_key
        }
        resp = requests.get(url, params=params, timeout=10)
        data = resp.json()
        if data["status"] != "OK":
            return []
        places = []
        for place in data["results"][:5]:
            places.append({
                "name": place["name"],
                "address": place.get("vicinity", ""),
                "rating": place.get("rating", 0),
                "lat": place["geometry"]["location"]["lat"],
```

```
                "lng": place["geometry"]["location"]["lng"],
                "wheelchair_accessible": place.get("wheelchair_accessible_entrance", False)
            })
        return places

navigation_agent = NavigationAgent()
```

4.2 routers/navigation.py

```
from fastapi import APIRouter, Query
from app.agents.navigation_agent import navigation_agent
from app.models import NavRouteRequest, NavRouteResponse

router = APIRouter()

@router.post("/route", response_model=NavRouteResponse)
async def get_route(request: NavRouteRequest):
    return navigation_agent.get_route(request)

@router.get("/nearby")
async def get_nearby(
    lat: float = Query(..., ge=-90, le=90),
    lng: float = Query(..., ge=-180, le=180),
    radius: int = Query(2000, ge=100, le=50000),
    type: str = Query("hospital")
):
    return {"places": navigation_agent.get_nearby(lat, lng, radius, type)}
```

4.3 Test: Navigation

```
# backend/tests/test_navigation.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_nav_route():
    payload = {
        "origin_lat": 28.6139,
        "origin_lng": 77.2090,
        "destination": "AIIMS Delhi",
        "mode": "walking"
    }
    response = client.post("/api/nav/route", json=payload)
    assert response.status_code == 200
    data = response.json()
    assert "distance" in data
    assert "duration" in data
    assert len(data["steps"]) > 0

def test_nav_nearby():
    response = client.get("/api/nav/nearby?lat=28.6139&lng=77.2090&type=hospital")
    assert response.status_code == 200
    data = response.json()
    assert "places" in data

def test_nav_invalid_coords():
    response = client.get("/api/nav/nearby?lat=999&lng=77.2090")
    assert response.status_code == 422
```

CHECKPOINT

Run pytest backend/tests/test_navigation.py -v. All 3 tests pass. Test /api/nav/route with real coords via Postman.

5 AUDIT AGENT (BACKEND) [30 min]

5.1 agents/audit_agent.py

```
import json
import google.generativeai as genai
from app.config import settings
from app.utils.prompts import AUDIT_PROMPT
from app.models import AuditResponse

genai.configure(api_key=settings.GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')

class AuditAgent:
    def analyze_accessibility(self, image_bytes: bytes) -> AuditResponse:
        image_part = {"mime_type": "image/jpeg", "data": image_bytes}
        response = model.generate_content([AUDIT_PROMPT, image_part])
        text = response.text.strip()
        for prefix in ["```json", "```"]:
            if text.startswith(prefix):
                text = text[len(prefix):]
        if text.endswith("```"):
            text = text[:-3]
        text = text.strip()
        data = json.loads(text)
        return AuditResponse(
            score=data.get("score", 50),
            issues=data.get("issues", []),
            fixes=data.get("fixes", [])
        )

audit_agent = AuditAgent()
```

5.2 routers/audit.py

```
from fastapi import APIRouter, UploadFile, File, HTTPException
from app.agents.audit_agent import audit_agent
from app.models import AuditResponse

router = APIRouter()

@router.post("/analyze", response_model=AuditResponse)
async def analyze_audit(file: UploadFile = File(...)):
    if file.content_type not in ["image/jpeg", "image/png", "image/jpg"]:
        raise HTTPException(status_code=400, detail="Only JPEG/PNG allowed")
    contents = await file.read()
    if len(contents) > 4 * 1024 * 1024:
        raise HTTPException(status_code=400, detail="Image must be under 4MB")
    try:
        result = audit_agent.analyze_accessibility(contents)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Audit failed: {str(e)}")
```

5.3 Test: Audit

```
# backend/tests/test_audit.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_audit_invalid_format():
    response = client.post("/api/audit/analyze",
        files={"file": ("test.txt", b"not image", "text/plain")})
    assert response.status_code == 400

# Integration test with real image:
```

```
# response = client.post("/api/audit/analyze",
#     files={"file": ("building.jpg", open("building.jpg","rb"), "image/jpeg")})
# assert response.status_code == 200
# assert 0 <= response.json()["score"] <= 100
# assert len(response.json()["issues"]) > 0
```

CHECKPOINT

Upload a building photo. Should return score 0-100 + issues + fixes.

6 CHAT ROUTER + FULL BACKEND TEST [20 min]

6.1 routers/chat.py

```
from fastapi import APIRouter
from app.agents.orchestrator import orchestrator
from app.models import ChatRequest, ChatResponse

router = APIRouter()

@router.post("", response_model=ChatResponse)
async def chat(request: ChatRequest):
    result = orchestrator.classify_intent(request.message)
    return ChatResponse(**result)
```

6.2 pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
asyncio_mode = auto
```

6.3 Run ALL Backend Tests

```
pytest backend/tests/ -v --tb=short

# Expected output:
# test_health.py::test_health_check PASSED
# test_chat.py::test_classify_vision_intent PASSED
# test_chat.py::test_classify_nav_intent PASSED
# test_chat.py::test_classify_audit_intent PASSED
# test_chat.py::test_classify_general_intent PASSED
# test_vision.py::test_vision_invalid_format PASSED
# test_vision.py::test_vision_no_file PASSED
# test_navigation.py::test_nav_route PASSED
# test_navigation.py::test_nav_nearby PASSED
# test_navigation.py::test_nav_invalid_coords PASSED
# test_audit.py::test_audit_invalid_format PASSED
```

CHECKPOINT

Run: pytest backend/tests/ -v. ALL tests should pass. If any fail, fix before moving on. Backend is now complete.

7 FRONTEND SCAFFOLD [30 min]

7.1 tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {
      colors: {
        slate: { 850: '#1e293b', 900: '#0f172a', 950: '#020617' },
        primary: { DEFAULT: '#f97316', hover: '#ea580c' }
      }
    }
  },
  plugins: []
}
```

7.2 index.html

```
<!DOCTYPE html>
<html lang="en" class="dark">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>AccessIndia AI</title>
</head>
<body class="bg-slate-900 text-white min-h-screen">
  <div id="root"></div>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/hands/hands.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/camera_utils/camera_utils.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/@mediapipe/drawing_utils/drawing_utils.js"></script>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

7.3 main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
)
```

7.4 index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body { @apply bg-slate-900 text-white antialiased; }
  button { @apply focus:outline-none focus:ring-2 focus:ring-primary; }
}

@layer components {
  .agent-card {
    @apply bg-slate-800 rounded-xl p-4 border border-slate-700
           hover:border-primary transition-all;
  }
  .chat-bubble-user {
```

```
    @apply bg-primary text-white rounded-2xl rounded-br-sm
           px-4 py-2 max-w-[80%];
  }
  .chat-bubble-agent {
    @apply bg-slate-700 text-white rounded-2xl rounded-bl-sm
           px-4 py-2 max-w-[80%];
  }
}
```

7.5 services/api.js

```
import axios from 'axios'

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000',
  timeout: 15000,
})

API.interceptors.response.use(
  (res) => res,
  (err) => {
    const msg = err.response?.data?.detail || 'Something went wrong.'
    return Promise.reject(new Error(msg))
  }
)

export const chatAPI = {
  sendMessage: (message) => API.post('/api/chat', { message }),
}

export const visionAPI = {
  analyzeImage: (file) => {
    const form = new FormData()
    form.append('file', file)
    return API.post('/api/vision/analyze', form, {
      headers: { 'Content-Type': 'multipart/form-data' }
    })
  }
}

export const navAPI = {
  getRoute: (data) => API.post('/api/nav/route', data),
  getNearby: (lat, lng, type = 'hospital') =>
    API.get(`/api/nav/nearby?lat=${lat}&lng=${lng}&type=${type}`),
}

export const auditAPI = {
  analyzeImage: (file) => {
    const form = new FormData()
    form.append('file', file)
    return API.post('/api/audit/analyze', form, {
      headers: { 'Content-Type': 'multipart/form-data' }
    })
  }
}

export default API
```

CHECKPOINT

Run: npm run dev. Frontend should load with dark background. No console errors.

8 ZUSTAND STORE + LAYOUT [30 min]

8.1 store/useAppStore.js

```
import { create } from 'zustand'

export const useAppStore = create((set, get) => ({
  activeAgent: 'orchestrator',
  messages: [],
  isLoading: false,
  loadingAgent: null,
  userLocation: { lat: null, lng: null },

  setActiveAgent: (agent) => set({ activeAgent: agent }),

  addMessage: (msg) => set((state) => ({
    messages: [...state.messages, {
      ...msg,
      id: crypto.randomUUID(),
      timestamp: new Date().toISOString()
    }]
  })),

  setLoading: (loading, agent = null) =>
    set({ isLoading: loading, loadingAgent: agent }),

  setUserLocation: (lat, lng) =>
    set({ userLocation: { lat, lng } }),

  clearChat: () => set({ messages: [] }),
}))
```

8.2 components/Layout/Sidebar.jsx

```
import React from 'react'
import { useAppStore } from '../../store/useAppStore'
import { Eye, MessageCircle, MapPin, ClipboardCheck, Bot } from 'lucide-react'

const agents = [
  { id: 'orchestrator', label: 'Chat', icon: Bot },
  { id: 'vision', label: 'Vision', icon: Eye },
  { id: 'communication', label: 'Comm', icon: MessageCircle },
  { id: 'navigation', label: 'Navigate', icon: MapPin },
  { id: 'audit', label: 'Audit', icon: ClipboardCheck },
]

export default function Sidebar() {
  const { activeAgent, setActiveAgent } = useAppStore()
  return (
    <nav className="w-16 md:w-60 bg-slate-800 border-r border-slate-700
                    flex flex-col h-screen">
      <div className="p-4 border-b border-slate-700">
        <h1 className="hidden md:block text-xl font-bold text-primary">
          AccessIndia
        </h1>
        <span className="md:hidden text-primary font-bold text-lg">AI</span>
      </div>
      <div className="flex-1 py-4 space-y-1">
        {agents.map((a) => {
          const Icon = a.icon
          const isActive = activeAgent === a.id
          return (
            <button
              key={a.id}
              onClick={() => setActiveAgent(a.id)}
              className={`w-full flex items-center gap-3 px-4 py-3
                transition-colors ${
```

```
              isActive
                ? 'bg-primary/20 text-primary border-r-2 border-primary'
                : 'text-slate-400 hover:text-white hover:bg-slate-700'
            }`}
            aria-label={a.label}
          >
            <Icon size={20} />
            <span className="hidden md:inline font-medium">{a.label}</span>
          </button>
        )
      })}
    </div>
  </nav>
 )
}
```

8.3 components/Layout/Header.jsx

```
import React from 'react'
import { useAppStore } from '../../store/useAppStore'
import { Accessibility } from 'lucide-react'

export default function Header() {
  const { activeAgent } = useAppStore()
  const titles = {
    orchestrator: 'AI Assistant',
    vision: 'Vision Agent',
    communication: 'Communication Agent',
    navigation: 'Navigation Agent',
    audit: 'Accessibility Audit',
  }
  return (
    <header className="h-16 bg-slate-800 border-b border-slate-700
                       flex items-center px-6">
      <Accessibility className="text-primary mr-3" size={24} />
      <div>
        <h2 className="text-lg font-semibold">{titles[activeAgent]}</h2>
        <p className="text-xs text-slate-400">
          AccessIndia AI -- Multi-Agent Accessibility Platform
        </p>
      </div>
    </header>
  )
}
```

8.4 App.jsx

```
import React from 'react'
import { Routes, Route } from 'react-router-dom'
import Sidebar from './components/Layout/Sidebar'
import Header from './components/Layout/Header'
import OrchestratorChat from './components/Agents/OrchestratorChat'
import VisionAgent from './components/Agents/VisionAgent'
import CommunicationAgent from './components/Agents/CommunicationAgent'
import NavigationAgent from './components/Agents/NavigationAgent'
import AuditAgent from './components/Agents/AuditAgent'

export default function App() {
  return (
    <div className="flex h-screen bg-slate-900">
      <Sidebar />
      <div className="flex-1 flex flex-col overflow-hidden">
        <Header />
        <main className="flex-1 overflow-y-auto p-6">
          <Routes>
            <Route path="/" element={<OrchestratorChat />} />
            <Route path="/vision" element={<VisionAgent />} />
            <Route path="/communication"
```

```
                element={<CommunicationAgent />} />
            <Route path="/navigation" element={<NavigationAgent />} />
            <Route path="/audit" element={<AuditAgent />} />
          </Routes>
        </main>
      </div>
    </div>
  )
}
```

CHECKPOINT

Click sidebar items. Routes should switch. Header title changes. No 404s.

9 SHARED COMPONENTS [30 min]

9.1 components/Shared/FileDrop.jsx

```
import React, { useCallback } from 'react'
import { useDropzone } from 'react-dropzone'
import { Upload } from 'lucide-react'

export default function FileDrop({ onFileSelect,
                                   accept = { 'image/*': [] } }) {
  const onDrop = useCallback((files) => {
    if (files[0]) onFileSelect(files[0])
  }, [onFileSelect])

  const { getRootProps, getInputProps, isDragActive } =
    useDropzone({ onDrop, accept, maxFiles: 1 })

  return (
    <div
      {...getRootProps()}
      className={`border-2 border-dashed rounded-xl p-8 text-center
        cursor-pointer transition-colors ${
        isDragActive
          ? 'border-primary bg-primary/10'
          : 'border-slate-600 hover:border-slate-400'
      }`}
    >
      <input {...getInputProps()} aria-label="Upload image" />
      <Upload className="mx-auto mb-2 text-slate-400" size={32} />
      <p className="text-sm text-slate-300">
        {isDragActive ? 'Drop image here' : 'Drag & drop or click to upload'}
      </p>
      <p className="text-xs text-slate-500 mt-1">JPEG, PNG (max 4MB)</p>
    </div>
  )
}
```

9.2 components/Shared/TTSButton.jsx

```
import React from 'react'
import { Volume2 } from 'lucide-react'

export default function TTSButton({ text, label = "Read aloud" }) {
  const speak = () => {
    if (!text) return
    window.speechSynthesis.cancel()
    const utterance = new SpeechSynthesisUtterance(text)
    utterance.lang = 'en-IN'
    utterance.rate = 1.0
    utterance.pitch = 1.0
    window.speechSynthesis.speak(utterance)
  }
  return (
    <button
      onClick={speak}
      className="inline-flex items-center gap-2 px-3 py-1.5
        bg-slate-700 hover:bg-slate-600 rounded-lg text-sm
        transition-colors"
      aria-label={label}
    >
      <Volume2 size={16} />
      <span>Listen</span>
    </button>
  )
}
```

9.3 components/Shared/LoadingAgent.jsx

```
import React from 'react'
import { Loader2 } from 'lucide-react'

export default function LoadingAgent({ agentName }) {
  return (
    <div className="flex items-center gap-3 text-slate-400 py-2">
      <Loader2 className="animate-spin text-primary" size={18} />
      <span className="text-sm">
        Routing to <span className="text-primary font-medium">
        {agentName}</span> Agent...
      </span>
    </div>
  )
}
```

9.4 components/Shared/CameraFeed.jsx

```
import React, { useRef, useState } from 'react'
import { Camera, CameraOff } from 'lucide-react'

export default function CameraFeed({ onFrame, width = 640, height = 480 }) {
  const videoRef = useRef(null)
  const [isActive, setIsActive] = useState(false)
  const [error, setError] = useState(null)

  const startCamera = async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia(
        { video: { width, height } })
      if (videoRef.current) {
        videoRef.current.srcObject = stream
        setIsActive(true)
        setError(null)
      }
    } catch (err) {
      setError('Camera access denied. Enable it in browser settings.')
      setIsActive(false)
    }
  }

  const stopCamera = () => {
    if (videoRef.current?.srcObject) {
      videoRef.current.srcObject.getTracks().forEach(t => t.stop())
      videoRef.current.srcObject = null
    }
    setIsActive(false)
  }

  return (
    <div className="relative">
      <div className="flex gap-2 mb-3">
        <button
          onClick={isActive ? stopCamera : startCamera}
          className={`flex items-center gap-2 px-4 py-2 rounded-lg
            font-medium transition-colors ${
            isActive
              ? 'bg-red-600 hover:bg-red-700'
              : 'bg-primary hover:bg-primary-hover'
          }`}
        >
          {isActive ? <CameraOff size={18} /> : <Camera size={18} />}
          {isActive ? 'Stop Camera' : 'Start Camera'}
        </button>
      </div>
      {error && <p className="text-red-400 text-sm mb-2">{error}</p>}
      <div className="relative rounded-xl overflow-hidden bg-slate-800
                      border border-slate-700"
```

```
            style={{ width, height }}>
          <video
            ref={videoRef}
            autoPlay playsInline muted
            className="w-full h-full object-cover"
            style={{ transform: 'scaleX(-1)' }}
          />
        </div>
      </div>
    )
}
```

CHECKPOINT

TTSButton: Click it. Should speak. CameraFeed: Click Start Camera. Should show webcam feed.

10 HOOKS [30 min]

10.1 hooks/useSpeechToText.js

```
import { useState, useRef, useCallback } from 'react'

export function useSpeechToText() {
  const [isListening, setIsListening] = useState(false)
  const [transcript, setTranscript] = useState('')
  const [error, setError] = useState(null)
  const recognitionRef = useRef(null)

  const startListening = useCallback(() => {
    const SpeechRecognition = window.SpeechRecognition
                  || window.webkitSpeechRecognition
    if (!SpeechRecognition) {
      setError('Speech recognition not supported.')
      return
    }
    setError(null)
    setTranscript('')
    const recognition = new SpeechRecognition()
    recognition.continuous = true
    recognition.interimResults = true
    recognition.lang = 'en-IN'
    recognition.onresult = (event) => {
      let final = '', interim = ''
      for (let i = event.resultIndex; i < event.results.length; i++) {
        if (event.results[i].isFinal)
          final += event.results[i][0].transcript
        else
          interim += event.results[i][0].transcript
      }
      setTranscript(final || interim)
    }
    recognition.onerror = (event) => {
      if (event.error !== 'no-speech') {
        setError(`Error: ${event.error}`)
      }
      setIsListening(false)
    }
    recognition.onend = () => setIsListening(false)
    recognition.start()
    recognitionRef.current = recognition
    setIsListening(true)
  }, [])

  const stopListening = useCallback(() => {
    recognitionRef.current?.stop()
    setIsListening(false)
  }, [])

  return { isListening, transcript, error, startListening, stopListening }
}
```

10.2 hooks/useTextToSpeech.js

```
import { useCallback } from 'react'

export function useTextToSpeech() {
  const speak = useCallback((text) => {
    if (!text) return
    window.speechSynthesis.cancel()
    const u = new SpeechSynthesisUtterance(text)
    u.lang = 'en-IN'
    u.rate = 1.0
    u.pitch = 1.0
```

```
    window.speechSynthesis.speak(u)
  }, [])

  const stop = useCallback(() => {
    window.speechSynthesis.cancel()
  }, [])

  return { speak, stop }
}
```

10.3 hooks/useGeolocation.js

```
import { useState, useEffect } from 'react'

export function useGeolocation() {
  const [location, setLocation] = useState(
    { lat: null, lng: null, error: null })

  useEffect(() => {
    if (!navigator.geolocation) {
      setLocation(prev => ({ ...prev, error: 'Not supported' }))
      return
    }
    navigator.geolocation.getCurrentPosition(
      (pos) => setLocation({
        lat: pos.coords.latitude,
        lng: pos.coords.longitude,
        error: null
      }),
      (err) => setLocation(prev => ({ ...prev, error: err.message })),
      { enableHighAccuracy: true, timeout: 10000 }
    )
  }, [])

  return location
}
```

10.4 Test: Hooks

```
// frontend/tests/useSpeechToText.test.js
import { describe, it, expect } from 'vitest'
import { renderHook, act } from '@testing-library/react'
import { useSpeechToText } from '../src/hooks/useSpeechToText'

describe('useSpeechToText', () => {
  it('initializes with correct defaults', () => {
    const { result } = renderHook(() => useSpeechToText())
    expect(result.current.isListening).toBe(false)
    expect(result.current.transcript).toBe('')
    expect(result.current.error).toBeNull()
  })
  it('sets error if SpeechRecognition unavailable', () => {
    const { result } = renderHook(() => useSpeechToText())
    act(() => result.current.startListening())
    expect(result.current.error).toContain('not supported')
  })
})

// frontend/tests/useTextToSpeech.test.js
import { useTextToSpeech } from '../src/hooks/useTextToSpeech'

describe('useTextToSpeech', () => {
  it('returns speak and stop functions', () => {
    const { result } = renderHook(() => useTextToSpeech())
    expect(typeof result.current.speak).toBe('function')
    expect(typeof result.current.stop).toBe('function')
  })
})
```

```
// Run: npx vitest
```

CHECKPOINT

Run: npx vitest. Hook tests should pass. Test useSpeechToText in browser: click mic, speak, see transcript.

11 GESTURE CLASSIFIER [20 min]

11.1 utils/gestureClassifier.js

```
const GESTURE_NAMES = [
  'hello', 'help', 'yes', 'no', 'thank_you',
  'eat', 'drink', 'bathroom', 'doctor', 'sorry'
]

// Pre-recorded templates (63 values per gesture, normalized 0-1)
// PLACEHOLDERS -- record real gestures before hackathon
const TEMPLATES = {
  hello: new Array(63).fill(0.5).map((v, i) => v + Math.sin(i) * 0.1),
  help: new Array(63).fill(0.5).map((v, i) => v + Math.cos(i) * 0.1),
  yes: new Array(63).fill(0.5).map((v, i) => v + (i % 3 === 0 ? 0.2 : 0)),
  no: new Array(63).fill(0.5).map((v, i) => v + (i % 3 === 1 ? 0.2 : 0)),
  thank_you: new Array(63).fill(0.5).map((v, i) =>
    v + (i % 5 === 0 ? 0.15 : 0)),
  eat: new Array(63).fill(0.5).map((v, i) =>
    v + Math.sin(i * 0.5) * 0.1),
  drink: new Array(63).fill(0.5).map((v, i) =>
    v + Math.cos(i * 0.5) * 0.1),
  bathroom: new Array(63).fill(0.5).map((v, i) =>
    v + (i > 30 ? 0.2 : 0)),
  doctor: new Array(63).fill(0.5).map((v, i) =>
    v + (i < 30 ? 0.2 : 0)),
  sorry: new Array(63).fill(0.5).map((v, i) =>
    v + Math.sin(i * 0.3) * 0.08),
}

function euclideanDistance(a, b) {
  let sum = 0
  for (let i = 0; i < a.length; i++) {
    sum += (a[i] - b[i]) ** 2
  }
  return Math.sqrt(sum)
}

export function classifyGesture(landmarks) {
  // landmarks: array of 21 {x,y,z} objects from MediaPipe
  const flat = landmarks.flatMap(lm => [lm.x, lm.y, lm.z])
  if (flat.length !== 63) {
    return { gesture: 'unknown', confidence: 0 }
  }
  let bestMatch = 'unknown', bestDist = Infinity
  for (const [name, template] of Object.entries(TEMPLATES)) {
    const dist = euclideanDistance(flat, template)
    if (dist < bestDist) {
      bestDist = dist
      bestMatch = name
    }
  }
  const THRESHOLD = 2.5
  const confidence = Math.max(0, 1 - bestDist / THRESHOLD)
  if (bestDist > THRESHOLD) {
    return { gesture: 'unknown', confidence: 0 }
  }
  return { gesture: bestMatch, confidence: parseFloat(confidence.toFixed(2)) }
}

export { GESTURE_NAMES }
```

11.2 Test: Gesture Classifier

```
// frontend/tests/gestureClassifier.test.js
import { describe, it, expect } from 'vitest'
import { classifyGesture, GESTURE_NAMES } from
```

```
  '../src/utils/gestureClassifier'

describe('classifyGesture', () => {
  it('returns unknown for empty input', () => {
    const result = classifyGesture([])
    expect(result.gesture).toBe('unknown')
    expect(result.confidence).toBe(0)
  })
  it('returns unknown for wrong landmark count', () => {
    const result = classifyGesture([{ x: 0.5, y: 0.5, z: 0 }])
    expect(result.gesture).toBe('unknown')
  })
  it('classifies a known gesture', () => {
    const landmarks = new Array(21).fill(0).map((_, i) => ({
      x: 0.5 + Math.sin(i) * 0.1,
      y: 0.5 + Math.sin(i) * 0.1,
      z: 0.5 + Math.sin(i) * 0.1,
    }))
    const result = classifyGesture(landmarks)
    expect(GESTURE_NAMES).toContain(result.gesture)
    expect(result.confidence).toBeGreaterThan(0)
    expect(result.confidence).toBeLessThanOrEqual(1)
  })
})

// Run: npx vitest
```

CHECKPOINT

Run npx vitest. All classifier tests pass. Record real gestures before hackathon by logging landmarks to console.

12 AGENT COMPONENTS (PART 1) [45 min]

12.1 components/Agents/OrchestratorChat.jsx

```
import React, { useState } from 'react'
import { useAppStore } from '../../store/useAppStore'
import { chatAPI } from '../../services/api'
import { Send, Mic } from 'lucide-react'
import LoadingAgent from '../Shared/LoadingAgent'
import TTSButton from '../Shared/TTSButton'
import { useSpeechToText } from '../../hooks/useSpeechToText'

export default function OrchestratorChat() {
  const [input, setInput] = useState('')
  const { messages, addMessage, isLoading, setLoading } = useAppStore()
  const { isListening, transcript, startListening, stopListening } =
    useSpeechToText()

  const handleSend = async () => {
    const text = input.trim() || transcript
    if (!text) return
    addMessage({ role: 'user', content: text, type: 'text' })
    setInput('')
    setLoading(true, 'orchestrator')
    try {
      const res = await chatAPI.sendMessage(text)
      const data = res.data
      addMessage({
        role: 'agent',
        agent: data.agent,
        content: data.message,
        type: 'text',
        metadata: { intent: data.intent, confidence: data.confidence }
      })
      if (data.agent !== 'general' && data.requires_upload) {
        addMessage({
          role: 'agent',
          content: `Switch to the ${data.agent} tab and upload an image.`,
          type: 'text'
        })
      }
    } catch (err) {
      addMessage({ role: 'agent', content: err.message, type: 'text' })
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="max-w-3xl mx-auto h-full flex flex-col">
      <div className="flex-1 overflow-y-auto space-y-4 mb-4 pr-2">
        {messages.length === 0 && (
          <div className="text-center text-slate-500 mt-20">
            <h3 className="text-xl font-semibold mb-2">
              Welcome to AccessIndia AI
            </h3>
            <p>Ask me anything or upload an image for analysis.</p>
          </div>
        )}
        {messages.map((msg) => (
          <div key={msg.id}
               className={`flex ${
                 msg.role === 'user' ? 'justify-end' : 'justify-start'
               }`}>
            <div className={msg.role === 'user'
              ? 'chat-bubble-user' : 'chat-bubble-agent'}>
```

```
              {msg.content}
              {msg.role === 'agent' && msg.agent && (
                <div className="mt-1 flex items-center gap-2">
                  <span className="text-xs text-primary font-medium">
                    {msg.agent} agent
                  </span>
                  <TTSButton text={msg.content} />
                </div>
              )}
            </div>
          </div>
        ))}
        {isLoading && (
          <LoadingAgent agentName={
            useAppStore.getState().loadingAgent} />
        )}
      </div>
      <div className="border-t border-slate-700 pt-4">
        <div className="flex gap-2">
          <button
            onClick={isListening ? stopListening : startListening}
            className={`p-3 rounded-xl transition-colors ${
              isListening ? 'bg-red-600' : 'bg-slate-700 hover:bg-slate-600'
            }`}
            aria-label={isListening ? 'Stop' : 'Start listening'}
          >
            <Mic size={20} />
          </button>
          <input
            type="text"
            value={input}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleSend()}
            placeholder={transcript || "Type your message..."}
            className="flex-1 bg-slate-800 border border-slate-600
              rounded-xl px-4 py-3 text-white placeholder-slate-500
              focus:border-primary focus:ring-1 focus:ring-primary"
          />
          <button
            onClick={handleSend}
            className="p-3 bg-primary hover:bg-primary-hover
              rounded-xl transition-colors"
            aria-label="Send"
          >
            <Send size={20} />
          </button>
        </div>
        {transcript && (
          <p className="text-xs text-slate-400 mt-1">
            Heard: {transcript}
          </p>
        )}
      </div>
    </div>
  )
}
```

CHECKPOINT

Type "I cant read this label" in chat. Should show routing to vision agent. TTS button should speak the response.

13 AGENT COMPONENTS (PART 2) [45 min]

13.1 components/Agents/VisionAgent.jsx

```
import React, { useState } from 'react'
import { visionAPI } from '../../services/api'
import FileDrop from '../Shared/FileDrop'
import TTSButton from '../Shared/TTSButton'
import { Eye, Loader2 } from 'lucide-react'

export default function VisionAgent() {
  const [result, setResult] = useState(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  const [preview, setPreview] = useState(null)

  const handleFile = async (file) => {
    setLoading(true)
    setError(null)
    setResult(null)
    setPreview(URL.createObjectURL(file))
    try {
      const res = await visionAPI.analyzeImage(file)
      setResult(res.data)
    } catch (err) {
      setError(err.message)
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="max-w-3xl mx-auto space-y-6">
      <div className="flex items-center gap-3 mb-4">
        <Eye className="text-primary" size={28} />
        <div>
          <h2 className="text-xl font-bold">Vision Agent</h2>
          <p className="text-sm text-slate-400">
            Upload an image to extract text, describe scenes, and identify objects.
          </p>
        </div>
      </div>

      <FileDrop onFileSelect={handleFile} />

      {preview && (
        <div className="rounded-xl overflow-hidden border border-slate-700">
          <img src={preview} alt="Uploaded preview"
               className="w-full max-h-64 object-contain bg-slate-800" />
        </div>
      )}

      {loading && (
        <div className="flex items-center gap-3 text-slate-400">
          <Loader2 className="animate-spin text-primary" size={20} />
          <span>Analyzing image...</span>
        </div>
      )}

      {error && (
        <div className="bg-red-900/30 border border-red-700 rounded-lg p-4 text-red-300">
          {error}
        </div>
      )}

      {result && (
```

```
        <div className="space-y-4">
          {result.ocr_text && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-2">Extracted Text</h3>
              <p className="text-slate-300 whitespace-pre-wrap">{result.ocr_text}</p>
              <div className="mt-3">
                <TTSButton text={result.ocr_text} label="Read extracted text" />
              </div>
            </div>
          )}
          {result.description && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-2">Scene Description</h3>
              <p className="text-slate-300">{result.description}</p>
              <div className="mt-3">
                <TTSButton text={result.description} label="Read description" />
              </div>
            </div>
          )}
          {result.detected_items?.length > 0 && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-2">Detected Objects</h3>
              <div className="flex flex-wrap gap-2">
                {result.detected_items.map((item, i) => (
                  <span key={i} className="px-3 py-1 bg-slate-700 rounded-full text-sm">
                    {item}
                  </span>
                ))}
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  )
}
```

13.2 components/Agents/CommunicationAgent.jsx

```
import React, { useState, useEffect, useRef } from 'react'
import { useSpeechToText } from '../../hooks/useSpeechToText'
import { useTextToSpeech } from '../../hooks/useTextToSpeech'
import CameraFeed from '../Shared/CameraFeed'
import { classifyGesture } from '../../utils/gestureClassifier'
import { Mic, MicOff, MessageSquare, Hand } from 'lucide-react'

export default function CommunicationAgent() {
  const [activeTab, setActiveTab] = useState('speech')
  const [detectedSign, setDetectedSign] = useState(null)
  const [signConfidence, setSignConfidence] = useState(0)
  const { isListening, transcript, error: sttError, startListening, stopListening } = useSpeechToText()
  const { speak } = useTextToSpeech()
  const handsRef = useRef(null)
  const cameraVideoRef = useRef(null)

  // Initialize MediaPipe Hands when sign tab is active
  useEffect(() => {
    if (activeTab !== 'sign' || !window.Hands) return
    const hands = new window.Hands({
      locateFile: (file) =>
        `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
    })
    hands.setOptions({
      maxNumHands: 1,
      modelComplexity: 1,
      minDetectionConfidence: 0.5,
      minTrackingConfidence: 0.5
```

```
  })
  hands.onResults((results) => {
    if (results.multiHandLandmarks && results.multiHandLandmarks.length > 0) {
      const landmarks = results.multiHandLandmarks[0]
      const result = classifyGesture(landmarks)
      setDetectedSign(result.gesture)
      setSignConfidence(result.confidence)
      if (result.gesture !== 'unknown' && result.confidence > 0.6) {
        speak(`You signed: ${result.gesture.replace('_', ' ')}`)
      }
    }
  })
  handsRef.current = hands
  return () => { handsRef.current = null }
}, [activeTab, speak])

return (
  <div className="max-w-3xl mx-auto space-y-6">
    <div className="flex items-center gap-3 mb-4">
      <MessageSquare className="text-primary" size={28} />
      <div>
        <h2 className="text-xl font-bold">Communication Agent</h2>
        <p className="text-sm text-slate-400">
          Speech-to-text, text-to-speech, and sign language detection.
        </p>
      </div>
    </div>

    <div className="flex gap-2 mb-4">
      <button
        onClick={() => setActiveTab('speech')}
        className={`px-4 py-2 rounded-lg font-medium transition-colors ${
          activeTab === 'speech'
            ? 'bg-primary text-white'
            : 'bg-slate-700 text-slate-300 hover:bg-slate-600'
        }`}
      >
        <Mic size={16} className="inline mr-2" />
        Speech
      </button>
      <button
        onClick={() => setActiveTab('sign')}
        className={`px-4 py-2 rounded-lg font-medium transition-colors ${
          activeTab === 'sign'
            ? 'bg-primary text-white'
            : 'bg-slate-700 text-slate-300 hover:bg-slate-600'
        }`}
      >
        <Hand size={16} className="inline mr-2" />
        Sign Language
      </button>
    </div>

    {activeTab === 'speech' && (
      <div className="space-y-4">
        <button
          onClick={isListening ? stopListening : startListening}
          className={`w-full py-4 rounded-xl font-bold text-lg transition-colors ${
            isListening
              ? 'bg-red-600 hover:bg-red-700'
              : 'bg-primary hover:bg-primary-hover'
          }`}
        >
          {isListening ? <MicOff size={24} className="inline mr-2" /> : <Mic size={24} className="inline mr-2" />}
          {isListening ? 'Stop Listening' : 'Start Listening'}
        </button>
        {sttError && (
```

```
            <div className="bg-red-900/30 border border-red-700 rounded-lg p-3 text-red-300 text-sm">
              {sttError}
            </div>
          )}
          {transcript && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-2">Transcript</h3>
              <p className="text-slate-300 text-lg">{transcript}</p>
            </div>
          )}
        </div>
      )}

      {activeTab === 'sign' && (
        <div className="space-y-4">
          <CameraFeed />
          {detectedSign && detectedSign !== 'unknown' && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-2">Detected Sign</h3>
              <p className="text-2xl font-bold text-white capitalize">
                {detectedSign.replace('_', ' ')}
              </p>
              <p className="text-sm text-slate-400">
                Confidence: {(signConfidence * 100).toFixed(0)}%
              </p>
            </div>
          )}
          <div className="text-xs text-slate-500">
            Supported signs: hello, help, yes, no, thank you, eat, drink, bathroom, doctor, sorry
          </div>
        </div>
      )}
    </div>
  )
}
```

CHECKPOINT

VisionAgent: Upload medicine image. Should show OCR + description + objects. CommunicationAgent: Test speech tab (mic) and sign tab (camera).

14 AGENT COMPONENTS (PART 3) [45 min]

14.1 components/Agents/NavigationAgent.jsx

```
import React, { useState, useEffect, useRef } from 'react'
import { navAPI } from '../../services/api'
import { useGeolocation } from '../../hooks/useGeolocation'
import { MapPin, Navigation, Search, Loader2 } from 'lucide-react'

export default function NavigationAgent() {
  const [destination, setDestination] = useState('')
  const [route, setRoute] = useState(null)
  const [nearby, setNearby] = useState([])
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  const location = useGeolocation()
  const mapRef = useRef(null)
  const mapInstanceRef = useRef(null)

  const handleSearch = async () => {
    if (!destination || !location.lat) return
    setLoading(true)
    setError(null)
    try {
      const [routeRes, nearbyRes] = await Promise.all([
        navAPI.getRoute({
          origin_lat: location.lat,
          origin_lng: location.lng,
          destination,
          mode: 'walking'
        }),
        navAPI.getNearby(location.lat, location.lng, 'hospital')
      ])
      setRoute(routeRes.data)
      setNearby(nearbyRes.data.places || [])
    } catch (err) {
      setError(err.message)
    } finally {
      setLoading(false)
    }
  }

  // Initialize Google Map
  useEffect(() => {
    if (!window.google?.maps || !location.lat) return
    const map = new window.google.maps.Map(mapRef.current, {
      center: { lat: location.lat, lng: location.lng },
      zoom: 14,
      styles: [
        { elementType: 'geometry', stylers: [{ color: '#1e293b' }] },
        { elementType: 'labels.text.fill', stylers: [{ color: '#94a3b8' }] },
        { elementType: 'labels.text.stroke', stylers: [{ color: '#0f172a' }] },
      ]
    })
    mapInstanceRef.current = map
    new window.google.maps.Marker({
      position: { lat: location.lat, lng: location.lng },
      map,
      title: 'You are here',
      icon: { url: 'http://maps.google.com/mapfiles/ms/icons/blue-dot.png' }
    })
  }, [location.lat, location.lng])

  return (
    <div className="max-w-4xl mx-auto space-y-6">
      <div className="flex items-center gap-3 mb-4">
```

```
  <Navigation className="text-primary" size={28} />
  <div>
    <h2 className="text-xl font-bold">Navigation Agent</h2>
    <p className="text-sm text-slate-400">
      Find wheelchair-friendly routes and nearby accessible facilities.
    </p>
  </div>
</div>

<div className="flex gap-2">
  <input
    type="text"
    value={destination}
    onChange={(e) => setDestination(e.target.value)}
    onKeyDown={(e) => e.key === 'Enter' && handleSearch()}
    placeholder="Search destination (e.g., AIIMS Hospital)"
    className="flex-1 bg-slate-800 border border-slate-600 rounded-xl px-4 py-3
      text-white placeholder-slate-500 focus:border-primary focus:ring-1 focus:ring-primary"
  />
  <button
    onClick={handleSearch}
    disabled={loading || !location.lat}
    className="px-6 py-3 bg-primary hover:bg-primary-hover rounded-xl
      font-medium transition-colors disabled:opacity-50 disabled:cursor-not-allowed"
  >
    {loading ? <Loader2 className="animate-spin" size={20} /> : <Search size={20} />}
  </button>
</div>

{location.error && (
  <div className="bg-amber-900/30 border border-amber-700 rounded-lg p-3 text-amber-300 text-sm">
    Location error: {location.error}. You can still search manually.
  </div>
)}

<div ref={mapRef} className="w-full h-80 rounded-xl border border-slate-700 bg-slate-800" />

{route && (
  <div className="agent-card">
    <h3 className="font-semibold text-primary mb-3">Route Details</h3>
    <div className="flex gap-6 mb-4">
      <div>
        <span className="text-slate-400 text-sm">Distance</span>
        <p className="text-xl font-bold">{route.distance}</p>
      </div>
      <div>
        <span className="text-slate-400 text-sm">Duration</span>
        <p className="text-xl font-bold">{route.duration}</p>
      </div>
    </div>
    <div className="space-y-2">
      {route.steps.map((step, i) => (
        <div key={i} className="flex items-start gap-3 p-3 bg-slate-700/50 rounded-lg">
          <span className="flex-shrink-0 w-6 h-6 bg-primary/20 text-primary rounded-full flex items-center justify-center text
            {i + 1}
          </span>
          <div>
            <p className="text-sm text-slate-300" dangerouslySetInnerHTML={{ __html: step.instruction }} />
            <span className="text-xs text-slate-500">{step.distance}</span>
          </div>
        </div>
      ))}
    </div>
  </div>
)}

{nearby.length > 0 && (
```

```
        <div className="agent-card">
          <h3 className="font-semibold text-primary mb-3">Nearby Accessible Facilities</h3>
          <div className="space-y-3">
            {nearby.map((place, i) => (
              <div key={i} className="flex items-center justify-between p-3 bg-slate-700/50 rounded-lg">
                <div>
                  <p className="font-medium">{place.name}</p>
                  <p className="text-xs text-slate-400">{place.address}</p>
                  {place.wheelchair_accessible && (
                    <span className="inline-block mt-1 px-2 py-0.5 bg-emerald-900/50 text-emerald-400 text-xs rounded-full">
                      Wheelchair Accessible
                    </span>
                  )}
                </div>
                <div className="text-right">
                  <span className="text-amber-400 font-bold">{place.rating}</span>
                  <MapPin size={16} className="inline ml-1 text-slate-500" />
                </div>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  )
}
```

14.2 components/Agents/AuditAgent.jsx

```
import React, { useState } from 'react'
import { auditAPI } from '../../services/api'
import FileDrop from '../Shared/FileDrop'
import { ClipboardCheck, Loader2, AlertTriangle, CheckCircle, Wrench } from 'lucide-react'

export default function AuditAgent() {
  const [result, setResult] = useState(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  const [preview, setPreview] = useState(null)

  const handleFile = async (file) => {
    setLoading(true)
    setError(null)
    setResult(null)
    setPreview(URL.createObjectURL(file))
    try {
      const res = await auditAPI.analyzeImage(file)
      setResult(res.data)
    } catch (err) {
      setError(err.message)
    } finally {
      setLoading(false)
    }
  }

  const getScoreColor = (score) => {
    if (score >= 71) return 'text-emerald-400'
    if (score >= 41) return 'text-amber-400'
    return 'text-red-400'
  }

  const getScoreBg = (score) => {
    if (score >= 71) return 'bg-emerald-900/30 border-emerald-700'
    if (score >= 41) return 'bg-amber-900/30 border-amber-700'
    return 'bg-red-900/30 border-red-700'
  }
```

```
return (
  <div className="max-w-3xl mx-auto space-y-6">
    <div className="flex items-center gap-3 mb-4">
      <ClipboardCheck className="text-primary" size={28} />
      <div>
        <h2 className="text-xl font-bold">Accessibility Audit</h2>
        <p className="text-sm text-slate-400">
          Upload a photo of a building or public space to get an accessibility score.
        </p>
      </div>
    </div>

    <FileDrop onFileSelect={handleFile} />

    {preview && (
      <div className="rounded-xl overflow-hidden border border-slate-700">
        <img src={preview} alt="Audit preview"
             className="w-full max-h-64 object-contain bg-slate-800" />
      </div>
    )}

    {loading && (
      <div className="flex items-center gap-3 text-slate-400">
        <Loader2 className="animate-spin text-primary" size={20} />
        <span>Analyzing accessibility...</span>
      </div>
    )}

    {error && (
      <div className="bg-red-900/30 border border-red-700 rounded-lg p-4 text-red-300">
        {error}
      </div>
    )}

    {result && (
      <div className="space-y-6">
        {/* Score Gauge */}
        <div className={`agent-card ${getScoreBg(result.score)}`}>
          <div className="flex items-center justify-between mb-4">
            <h3 className="font-semibold text-lg">Accessibility Score</h3>
            <span className={`text-4xl font-bold ${getScoreColor(result.score)}`}>
              {result.score}/100
            </span>
          </div>
          <div className="w-full bg-slate-700 rounded-full h-4 overflow-hidden">
            <div
              className={`h-full rounded-full transition-all duration-1000 ${
                result.score >= 71 ? 'bg-emerald-500' :
                result.score >= 41 ? 'bg-amber-500' : 'bg-red-500'
              }`}
              style={{ width: `${result.score}%` }}
            />
          </div>
          <p className="text-sm text-slate-400 mt-2">
            {result.score >= 71 ? 'Good accessibility' :
             result.score >= 41 ? 'Needs improvement' : 'Poor accessibility'}
          </p>
        </div>

        {/* Issues */}
        {result.issues?.length > 0 && (
          <div className="agent-card">
            <h3 className="font-semibold text-primary mb-3 flex items-center gap-2">
              <AlertTriangle size={18} />
              Issues Found
            </h3>
            <div className="space-y-3">
```

```
                {result.issues.map((issue, i) => (
                  <div key={i} className="p-3 bg-slate-700/50 rounded-lg">
                    <div className="flex items-center gap-2 mb-1">
                      <span className={`px-2 py-0.5 rounded text-xs font-medium ${
                        issue.severity === 'high' ? 'bg-red-900/50 text-red-400' :
                        issue.severity === 'medium' ? 'bg-amber-900/50 text-amber-400' :
                        'bg-blue-900/50 text-blue-400'
                      }`}>
                        {issue.severity}
                      </span>
                      <span className="font-medium text-sm">{issue.category}</span>
                    </div>
                    <p className="text-sm text-slate-400">{issue.description}</p>
                  </div>
                ))}
              </div>
            </div>
          )}

          {/* Fixes */}
          {result.fixes?.length > 0 && (
            <div className="agent-card">
              <h3 className="font-semibold text-primary mb-3 flex items-center gap-2">
                <Wrench size={18} />
                Recommended Fixes
              </h3>
              <div className="space-y-3">
                {result.fixes.map((fix, i) => (
                  <div key={i} className="p-3 bg-slate-700/50 rounded-lg flex items-start gap-3">
                    <span className="flex-shrink-0 w-6 h-6 bg-primary/20 text-primary rounded-full flex items-center justify-center
                      {fix.priority}
                    </span>
                    <div>
                      <p className="text-sm">{fix.action}</p>
                      <span className="text-xs text-slate-500">Cost: {fix.estimated_cost}</span>
                    </div>
                  </div>
                ))}
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  )
}
```

CHECKPOINT

NavigationAgent: Search "hospital" -> shows map + route + nearby. AuditAgent: Upload building photo -> shows animated score + issues + fixes.

15 INTEGRATION + POLISH [2 hours]

15.1 Wire Everything Together

- Ensure all imports are correct (no broken paths)
- Add error boundaries around each agent component
- Test switching between all 5 tabs rapidly
- Verify dark theme consistency across ALL pages

15.2 Add Loading States

- Every async operation shows a loading indicator
- No button should be clickable while loading
- Skeleton screens preferred over spinners for content

15.3 Error Handling

- Graceful fallbacks for: no camera, no mic, no location, no internet
- User-friendly error messages (not raw API errors)
- Retry buttons on all error states

15.4 Mobile Responsiveness

- Test on phone or Chrome DevTools mobile view
- Sidebar collapses to icons on mobile
- Chat input stays fixed at bottom
- Maps and images scale correctly

15.5 Accessibility Audit of Your Own App

- All buttons have aria-label
- Color contrast WCAG AA (4.5:1 minimum)
- Keyboard navigation works (Tab, Enter, Escape)
- Focus rings visible on all interactive elements

CHECKPOINT

Test entire app on mobile. All 5 tabs work. No console errors. All async operations have loading states.

16 TESTING + DEMO PREP [2 hours]

16.1 Frontend Tests

```
// frontend/tests/App.test.jsx
import { describe, it, expect } from 'vitest'
import { render, screen } from '@testing-library/react'
import { BrowserRouter } from 'react-router-dom'
import App from '../src/App'

describe('App', () => {
  it('renders sidebar with all agents', () => {
    render(<BrowserRouter><App /></BrowserRouter>)
    expect(screen.getByText('AccessIndia')).toBeInTheDocument()
    expect(screen.getByLabelText('Chat')).toBeInTheDocument()
    expect(screen.getByLabelText('Vision')).toBeInTheDocument()
  })
})

// Run: npx vitest --run
```

16.2 End-to-End Test Script

```
1. Open app -> see dark dashboard
2. Click Vision tab -> upload medicine.jpg -> see OCR + description + objects
3. Click Communication tab -> click mic -> speak "hello" -> see transcript
4. Click Navigation tab -> search "hospital" -> see map + route + nearby
5. Click Audit tab -> upload building.jpg -> see score + issues + fixes
6. Click Chat tab -> type "I cant read this" -> see routing to vision
7. Test TTS on all text outputs -> should speak
8. Test on mobile view -> all tabs accessible
```

16.3 Record Demo Video (Backup)

```
Use OBS or screen recorder:
- 2 minutes max
- Show all 4 agents working
- Voiceover explaining each feature
- Upload to YouTube unlisted as backup
```

16.4 Prepare Fallback Data

```
Create 3 pre-generated responses in case APIs fail:
- vision_fallback.json (medicine label)
- audit_fallback.json (building entrance)
- nav_fallback.json (hospital route)
Store in frontend/src/data/ and use when API returns 500
```

CHECKPOINT

All tests pass. Demo video recorded. Fallback data ready. No P0 bugs remaining.

17 DEPLOYMENT [1 hour]

17.1 Backend Deploy (Render)

```
1. Push backend to GitHub
2. Go to render.com -> New Web Service
3. Connect GitHub repo, select backend folder
4. Build Command: pip install -r requirements.txt
5. Start Command: uvicorn app.main:app --host 0.0.0.0 --port $PORT
6. Add Environment Variables from .env
7. Deploy and copy URL
```

17.2 Frontend Deploy (Vercel)

```
1. Push frontend to GitHub (same or separate repo)
2. Go to vercel.com -> New Project
3. Connect GitHub repo, select frontend folder
4. Framework Preset: Vite
5. Build Command: npm run build
6. Output Directory: dist
7. Add Environment Variables: VITE_API_URL=your-render-url
8. Deploy and copy URL
```

17.3 Update CORS

```
# After deploying frontend, update backend .env:
CORS_ORIGINS=https://your-frontend.vercel.app,http://localhost:5173

# Redeploy backend
```

17.4 Final Verification

- Open frontend URL -> no console errors
- Test chat -> backend responds
- Test vision upload -> image analyzes
- Test navigation -> map loads
- Test audit -> score appears

CHECKPOINT

Both URLs live. All features work in production. Submit prototype link + pitch deck.

TROUBLESHOOTING GUIDE

Backend Issues

Symptom	Fix
ModuleNotFoundError	Run: pip install -r requirements.txt
CORS error in browser	Update CORS_ORIGINS in .env with frontend URL
Gemini API 429	Wait 60s, retry. Or use fallback mock data.
Google Maps 403	Check API key restrictions. Enable Directions + Places APIs.
Port already in use	Kill process: lsof -ti:8000 | xargs kill -9
Pydantic validation error	Check request JSON matches models.py exactly.

Frontend Issues

Symptom	Fix
Blank white screen	Check console for import errors. Verify all file paths.
CORS error	Backend CORS_ORIGINS must include exact frontend URL.
Map not loading	Check VITE_GOOGLE_MAPS_KEY. Enable Maps JS API.
Mic not working	Use HTTPS or localhost. Chrome only. Check permissions.
Camera not working	Use HTTPS or localhost. Check browser permissions.
Dark theme not applied	Verify tailwind.config.js content paths include all files.
Build fails on Vercel	Check package.json has "build": "vite build" script.

QUICK REFERENCE

Useful Commands

```
# Backend
python -m app.main          # Start dev server
pytest tests/ -v            # Run all tests
pytest tests/test_chat.py -v -k vision   # Run specific test

# Frontend
npm run dev                 # Start dev server
npm run build               # Production build
npx vitest                  # Run tests in watch mode
npx vitest --run            # Run tests once

# Git
git add . && git commit -m "checkpoint"
git push origin main
```

API Endpoints Summary

```
POST /api/chat              # Intent classification
POST /api/vision/analyze    # Image -> OCR + description
POST /api/nav/route         # Get walking directions
GET  /api/nav/nearby        # Find nearby places
POST /api/audit/analyze     # Image -> accessibility score
GET  /health                # Health check
```

Environment Variables

```
# Backend (.env)
GEMINI_API_KEY=...
GOOGLE_MAPS_API_KEY=...
CORS_ORIGINS=...
BACKEND_PORT=8000

# Frontend (.env)
VITE_API_URL=...
VITE_GOOGLE_MAPS_KEY=...
```

Supported Gestures

```
hello, help, yes, no, thank_you, eat, drink, bathroom, doctor, sorry
```

LET'S BUILD IT.

Team Coin Hustlers

HackAgentAIx 2026

"Accessibility is not a privilege. It is a fundamental right."

Build Guide v2.0 | 26 July 2026